USTO-MB

Faculté des Mathématiques et Informatique

Département d'Informatique

Guide Complet MongoDB & REST APIs

Module 1 : Introduction à MongoDB

1.1 Qu'est-ce que MongoDB ?

- MongoDB est une base de données NoSQL orientée documents
- Au lieu de tables (SQL), on a des COLLECTIONS
- Au lieu de lignes (SQL), on a des DOCUMENTS JSON

1.2 Types de données MongoDB

```
{
    _id: ObjectId("507f1f77bcf86cd799439011"),  // Identifiant unique auto-généré
    nom: "Marie Dupont",                        // String
    age: 22,                                    // Number (int)
    moyenne: 15.5,                              // Number (double)
    actif: true,                                // Boolean
    dateInscription: new Date("2024-01-15"),    // Date
    adresse: {                                  // Document imbriqué (Embedded)
        rue: "12 rue de Paris",
        ville: "Lyon"
    },
    competences: ["JavaScript", "Python", "SQL"], // Array
    notes: null,                                // Null
    metadata: undefined                         // Évitez undefined (non stocké)
}
```

Module 2 : CRUD Operations (30 min)

2.1 CREATE - Créer des documents

insertOne() - Insérer un seul document

```
// Syntaxe de base
db.collection.insertOne(document)

// EXEMPLE PRATIQUE :
// On veut ajouter un nouvel étudiant dans la collection "etudiants"
db.etudiants.insertOne({
    // _id est auto-généré si non spécifié
    nom: "Alice Martin",           // Champ string
    age: 21,                        // Champ number
    email: "alice@univ.fr",         // Champ string
    filiere: "Informatique",        // Champ string
    notes: [12, 15, 18],           // Array de numbers
    dateInscription: new Date(),    // Date actuelle
    boursier: false                 // Boolean
})

// RETOUR (résultat) :
{
    "acknowledged": true,           // Opération réussie
    "insertedId": ObjectId("...")  // ID du nouveau document
}
```

insertMany() - Insérer plusieurs documents

```
// Syntaxe
db.collection.insertMany([doc1, doc2, ...])
// EXEMPLE :
db.etudiants.insertMany([
    {
        nom: "Bob Chen",
        age: 23,
        email: "bob@univ.fr",
        filiere: "Mathématiques",
        notes: [14, 16, 15],
        dateInscription: new Date("2024-01-10"),
        boursier: true
    },
    {
        nom: "Carla Silva",
        age: 20,
        email: "carla@univ.fr",
        filiere: "Informatique",
        notes: [18, 19, 17],
        dateInscription: new Date("2024-01-12"),
        boursier: false
    }
])
// ASTUCE : Utilisez insertMany pour les données de test
// C'est beaucoup plus rapide que insertOne en boucle
```

2.2 READ - Lire des documents

find() - Rechercher plusieurs documents

```
// Syntaxe complète
db.collection.find(filtre, projection)
// EXEMPLE 1 : Tous les étudiants
db.etudiants.find({})
// {} = filtre vide = tous les documents
// Retourne un CURSEUR (itérateur), pas un tableau
// EXEMPLE 2 : Étudiants en informatique
db.etudiants.find({ filiere: "Informatique" })
// { filiere: "Informatique" } = clause WHERE filiere = 'Informatique'
// EXEMPLE 3 : Étudiants majeurs (>= 18 ans)
db.etudiants.find({ age: { $gte: 18 } })
// $gte = Greater Than or Equal (>=)
// Opérateurs de comparaison : $eq, $ne, $gt, $gte, $lt, $lte
// EXEMPLE 4 : Étudiants boursiers en informatique
db.etudiants.find({
    filiere: "Informatique",
    boursier: true
})
// Les conditions sont en AND implicite (toutes doivent être vraies)
```

Opérateurs logiques

```
// $or - Au moins une condition vraie
db.etudiants.find({
    $or: [
        { filiere: "Informatique" },
        { filiere: "Mathématiques" }
    ]
})
// $and - Toutes les conditions vraies (explicite)
db.etudiants.find({
    $and: [
        { age: { $gte: 20 } },
        { age: { $lte: 25 } }
    ]
})// Équivalent à : { age: { $gte: 20, $lte: 25 } }
// $in - Dans une liste de valeurs
db.etudiants.find({ filiere: { $in: ["Informatique", "Mathématiques"]
    } })
// $nin - PAS dans une liste
db.etudiants.find({ age: { $nin: [20, 21, 22] } })
// $not - Négation
db.etudiants.find({ boursier: { $not: { $eq: true } } })
```

Projection (choisir les champs)

```
// Syntaxe : find(filtre, { champ: 1, champ: 0 })
// 1 = inclure, 0 = exclure
// _id est toujours inclus par défaut

// EXEMPLE : Noms et emails uniquement
db.etudiants.find(
    { filiere: "Informatique" },     // Filtre
    { nom: 1, email: 1, _id: 0 }     // Projection
)
// Retourne : { nom: "Alice", email: "alice@univ.fr" }

// EXEMPLE : Tout sauf les notes
db.etudiants.find(
    {},
    { notes: 0 }                      // Exclure le champ notes
)
```

findOne() - Un seul document

```
// Retourne le PREMIER document correspondant (ou null)
db.etudiants.findOne({ email: "alice@univ.fr" })

// Avec projection
db.etudiants.findOne(
    { _id: ObjectId("507f1f77bcf86cd799439011") },
    { nom: 1, notes: 1 }
)
```

Méthodes de curseur

```
// find() retourne un curseur, on peut chaîner :

db.etudiants.find({})
    .limit(5)           // Limiter à 5 résultats
    .skip(10)           // Sauter les 10 premiers (pagination)
    .sort({ age: -1 })  // Trier par age décroissant (-1), croissant
        (1)
    .count()            // Compter les documents

// EXEMPLE : Pagination classique (page 2, 10 items par page)
const page = 2;
const limit = 10;
db.etudiants.find({})
    .skip((page - 1) * limit)  // Skip 10
    .limit(limit)              // Limit 10
    .sort({ nom: 1 })          // Tri alphabétique
```

2.3 UPDATE - Modifier des documents

updateOne() - Modifier un document

```
// Syntaxe
db.collection.updateOne(filtre, modification, options)

// EXEMPLE 1 : Modifier l'âge d'Alice
db.etudiants.updateOne(
    { nom: "Alice Martin" },           // Filtre : trouver Alice
    { $set: { age: 22 } }              // $set = définir/modifier un
        champ
)
// $set crée le champ s'il n'existe pas, le modifie sinon

// EXEMPLE 2 : Ajouter un champ
db.etudiants.updateOne(
    { nom: "Alice Martin" },
    { $set: { telephone: "0612345678" } }  // Nouveau champ
)

// EXEMPLE 3 : Modifier plusieurs champs
db.etudiants.updateOne(
    { _id: ObjectId("...") },
    {
        $set: {
```

```
    {
        $set: {
            age: 23,
            filiere: "Data Science",
            derniereMaj: new Date()
        }
    }
)
```

Opérateurs d'update importants

```
// $unset - Supprimer un champ
db.etudiants.updateOne(
    { nom: "Alice Martin" },
    { $unset: { telephone: "" } }      // Supprime le champ telephone
)

// $inc - Incrémenter (pour les nombres)
db.etudiants.updateOne(
    { nom: "Alice Martin" },
    { $inc: { age: 1 } }               // age = age + 1
)
// Utile pour : compteurs, vues, likes, stock...

// $push - Ajouter à un array
db.etudiants.updateOne(
    { nom: "Alice Martin" },
    { $push: { notes: 16 } }           // Ajoute 16 au tableau notes
)

// $push avec $each - Ajouter plusieurs éléments
db.etudiants.updateOne(
```

```
db.etudiants.updateOne(
    { nom: "Alice Martin" },
    { $push: { notes: { $each: [14, 17] } } }
)

// $pull - Retirer d'un array
db.etudiants.updateOne(
    { nom: "Alice Martin" },
    { $pull: { notes: 12 } }           // Retire la valeur 12
)

// $addToSet - Ajouter si pas déjà présent (ensemble)
db.etudiants.updateOne(
    { nom: "Alice Martin" },
    { $addToSet: { competences: "MongoDB" } }  // Pas de doublon
)
```

updateMany() - Modifier plusieurs documents

```
// Augmenter l'âge de tous les étudiants en informatique
db.etudiants.updateMany(
    { filiere: "Informatique" },       // Filtre : tous les info
    { $inc: { age: 1 } }               // Incrémente l'âge de 1
)

// Mettre à jour tous les boursiers
db.etudiants.updateMany(
    { boursier: true },
    { $set: { statut: "prioritaire" } }
)

// ATTENTION : Sans filtre, modifie TOUT !
db.etudiants.updateMany(
    {},                                // Tous les documents !
    { $set: { universite: "Lyon 1" } }
)
```

replaceOne() - Remplacer complètement

```
// Remplace TOUT le document (sauf _id)
db.etudiants.replaceOne(
    { nom: "Alice Martin" },
    {
        nom: "Alice Martin",
        age: 25,
        nouveauChamp: "valeur"
        // Les anciens champs (email, notes...) sont SUPPRIMÉS
    }
)
```

2.4 DELETE - Supprimer des documents

```
// deleteOne() - Supprimer UN document
db.etudiants.deleteOne({ nom: "Bob Chen" })
// Supprime le premier Bob Chen trouvé

// deleteMany() - Supprimer PLUSIEURS documents
db.etudiants.deleteMany({ filiere: "Ancienne Filiere" })

// ATTENTION DANGER : Supprimer tous les documents
db.etudiants.deleteMany({})  // Collection vide mais existe encore

// drop() - Supprimer toute la collection
db.etudiants.drop()            // Supprime collection + index

// EXEMPLE SÉCURISÉ : Supprimer les étudiants inactifs depuis 2 ans
db.etudiants.deleteMany({
    derniereConnexion: { $lt: new Date("2022-01-01") }
})
```

2.5 Récap CRUD

Opération	Méthode	Usage
Create	insertOne()	1 document
	insertMany()	N documents
Read	find()	Recherche multiple
	findOne()	Recherche unique
Update	updateOne()	Modifier 1 doc
	updateMany()	Modifier N docs
	replaceOne()	Remplacer 1 doc
Delete	deleteOne()	Supprimer 1 doc
	deleteMany()	Supprimer N docs

Module 3 : Aggregation Framework (25 min)

3.1 Concept du Pipeline

```
// L'aggregation est un PIPELINE (tuyau) de traitement
// Les documents passent par plusieurs STAGES (étapes)
// Chaque stage transforme les données

// Structure visuelle :
//Documents (Collection) =>   Stage 1 : $match (Filtrer) => Stage 2 :
    $group (Agréger) => Stage 3 : $sort (Trier)

// Syntaxe de base
db.collection.aggregate([
    { $stage1: { ... } },    // Étape 1
    { $stage2: { ... } },    // Étape 2
    { $stage3: { ... } }     // Étape 3
])// Le résultat d'une étape est l'entrée de la suivante
```

3.2 Stages essentiels

$match - Filtrer (comme WHERE)

```
// EXEMPLE : Pipeline simple
db.etudiants.aggregate([
    {
        $match: {
            filiere: "Informatique",    // Garde uniquement les info
            age: { $gte: 20 }            // Qui ont 20 ans ou plus
        }
    }
])
// Équivalent à : db.etudiants.find({ filiere: "Informatique", age: { $gte: 20 } })

// BONNE PRATIQUE : Mettre $match en PREMIER pour réduire les données tôt
```

$project - Projection (choisir/renommer/calculer)

```
db.etudiants.aggregate([
    { $match: { filiere: "Informatique" } },
    {
        $project: {
            _id: 0,                         // Exclure l'ID
            nom: 1,                         // Garder le nom
            email: 1,                       // Garder l'email
            anneeNaissance: {               // NOUVEAU champ calculé
                $subtract: [2024, "$age"]   // 2024 - age
            },
            categorie: {                    // Champ conditionnel
                $cond: {
                    if: { $gte: ["$age", 21] },
                    then: "Majeur",
                    else: "Mineur"
                }
            }
        }
    }
])

// Résultat :
```

```
// Résultat :
// { nom: "Alice", email: "alice@univ.fr", anneeNaissance: 2002,
    categorie: "Majeur" }
```

$group - Agréger (GROUP BY)

```
// EXEMPLE 1 : Compter par filière
db.etudiants.aggregate([
    {
        $group: {
            _id: "$filiere",            // Grouper par filière
            total: { $sum: 1 },         // Compter ($sum: 1 = +1 par
                doc)
            ageMoyen: { $avg: "$age" }, // Moyenne d'âge
            plusJeune: { $min: "$age" }, // Âge minimum
            plusVieux: { $max: "$age" }  // Âge maximum
        }
    }
])

// Résultat :
// [
//   { _id: "Informatique", total: 45, ageMoyen: 22.5, plusJeune: 19,
    plusVieux: 28 },
//   { _id: "Mathématiques", total: 30, ageMoyen: 23.1, plusJeune: 20
    , plusVieux: 30 }
// ]
```

```

// EXEMPLE 2 : Agrégation sur arrays (unwind)
db.etudiants.aggregate([
    { $unwind: "$notes" },              // Découpe le tableau notes
    {                                   // Crée un doc par note
        $group: {
            _id: "$nom",
            moyenne: { $avg: "$notes" }, // Calcule la moyenne
            nbNotes: { $sum: 1 }         // Compte les notes
        }
    }
])
// $unwind transforme [12, 15, 18] en 3 documents séparés
```

$sort, $limit, $skip

```
db.etudiants.aggregate([
    { $match: { filiere: "Informatique" } },
    {
        $project: {
            nom: 1,
            moyenne: { $avg: "$notes" }     // Calculer moyenne
        }
    },
    { $sort: { moyenne: -1 } },             // Trier par moyenne DESC
    { $skip: 5 },                           // Sauter top 5
    { $limit: 10 }                          // Prendre les 10 suivants
])
// Équivalent à : "Top 10-15 des étudiants info par moyenne"
```

3.3 Pipeline complet avancé

```
// SCÉNARIO : Classement des filières par nombre d'étudiants boursiers
// avec moyenne d'âge, trié par nombre décroissant

db.etudiants.aggregate([
    // ÉTAPE 1 : Filtrer uniquement les boursiers
    {
        $match: { boursier: true }
    },

    // ÉTAPE 2 : Grouper par filière et calculer stats
    {
        $group: {
            _id: "$filiere",
            nbBoursiers: { $sum: 1 },
            ageMoyen: { $avg: "$age" },
            etudiants: { $push: "$nom" }    // Array des noms
        }
    },

    // ÉTAPE 3 : Ajouter un champ calculé (densité)
```

```
21      {
22          $addFields: {
23              categorieAge: {
24                  $cond: {
25                      if: { $gt: ["$ageMoyen", 22] },
26                      then: "Senior",
27                      else: "Junior"
28                  }
29              }
30          }
31      },
32
33      // ÉTAPE 4 : Filtrer les filières avec + de 5 boursiers
34      {
35          $match: { nbBoursiers: { $gte: 5 } }
36      },
37
38      // ÉTAPE 5 : Trier par nombre de boursiers décroissant
39      {
40          $sort: { nbBoursiers: -1 }
41      },
```

```
42
43      // ÉTAPE 6 : Formater le résultat final
44      {
45          $project: {
46              _id: 0,
47              filiere: "$_id",
48              nbBoursiers: 1,
49              ageMoyen: { $round: ["$ageMoyen", 1] },  // Arrondi 1
                    décimale
50              categorieAge: 1,
51              exempleEtudiants: { $slice: ["$etudiants", 3] }  // 3
                    premiers noms
52          }
53      }
54  ])
```

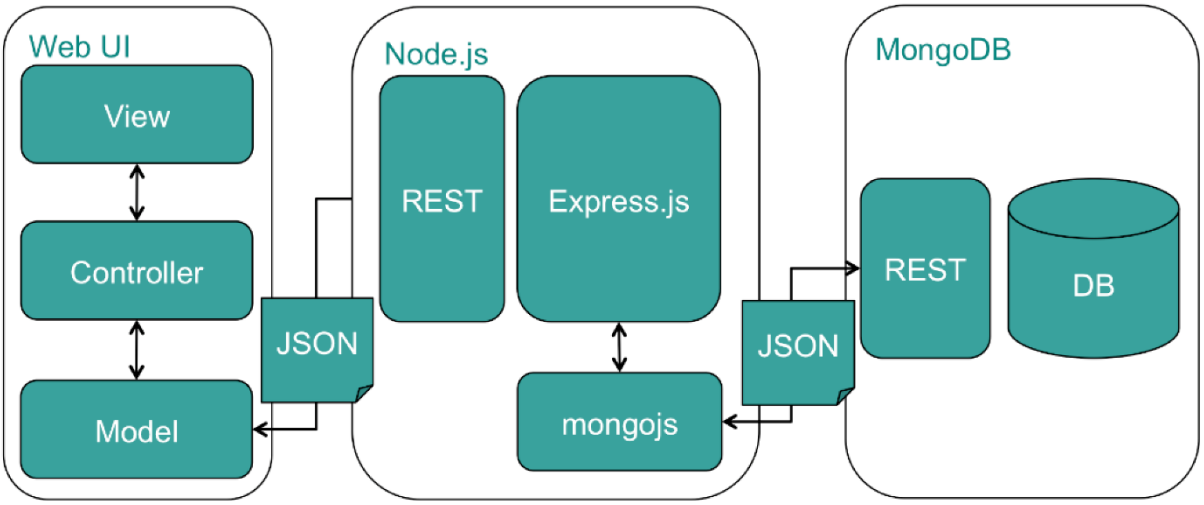
3.4 Jointures avec $lookup

```
// Structure : Deux collections
// etudiants : { _id, nom, filiere_id }
// filieres : { _id, nom, departement }

// ÉQUIVALENT SQL : SELECT * FROM etudiants JOIN filieres ON
    etudiants.filiere_id = filieres._id

db.etudiants.aggregate([
    {
        $lookup: {
            from: "filieres",           // Collection à joindre
            localField: "filiere_id",     // Champ dans etudiants
            foreignField: "_id",          // Champ dans filieres
            as: "filiere_info"            // Nom du nouveau champ
                array
        }
    },
    // filiere_info est un ARRAY (même si 1 seul résultat)
    {
        $unwind: "$filiere_info"        // Convertit array en objet
    },
```

```
        $unwind: "$filiere_info"        // Convertit array en objet
    },
    {
        $project: {
            nom: 1,
            filiere: "$filiere_info.nom",
            departement: "$filiere_info.departement"
        }
    }
])
```

Module 4 : REST APIs avec MongoDB (20 min)

4.1 Architecture



4.2 Setup Node.js/Express/Mongoose

```
1  // ============ server.js ============
2
3  // 1. IMPORTS
4  const express = require('express');       // Framework web
5  const mongoose = require('mongoose');     // ODM MongoDB
6  const app = express();
7
8  // 2. MIDDLEWARE (traitement des requêtes)
9  app.use(express.json());                  // Parser le JSON des
       requêtes
10
11 // 3. CONNEXION MONGODB
12 mongoose.connect('mongodb://localhost:27017/universite')
13     .then(() => console.log('Connecté à MongoDB'))
14     .catch(err => console.error('Erreur connexion:', err));
15
16 // 4. DÉFINITION DU MODÈLE (Schéma)
17 const etudiantSchema = new mongoose.Schema({
18     nom: { type: String, required: true },
19     email: { type: String, required: true, unique: true },
20     age: { type: Number, min: 0 },
```

```
20      age: { type: Number, min: 0 },
21      filiere: String,
22      notes: [Number],
23      dateInscription: { type: Date, default: Date.now },
24      actif: { type: Boolean, default: true }
25  });
26
27  const Etudiant = mongoose.model('Etudiant', etudiantSchema);
28  // 'Etudiant' → collection 'etudiants' (pluralisé automatiquement)
29
30  // 5. LANCEMENT SERVEUR
31  app.listen(3000, () => console.log('Serveur sur port 3000'));
```

4.3 CRUD API Complet

CREATE - POST /etudiants

```
1   // Créer un étudiant
2   app.post('/etudiants', async (req, res) => {
3       try {
4           // req.body contient les données JSON envoyées par le client
5           const nouvelEtudiant = new Etudiant(req.body);
6
7           // Sauvegarde dans MongoDB (équivalent insertOne)
8           const sauvegarde = await nouvelEtudiant.save();
9
10          // Réponse 201 Created avec le document créé
11          res.status(201).json({
12              message: 'Étudiant créé',
13              data: sauvegarde
14          });
15      } catch (erreur) {
16          // 400 Bad Request si validation échoue
17          res.status(400).json({ erreur: erreur.message });
18      }
19  });
20
21  // EXEMPLE D'APPEL (avec curl ou Postman) :
```

```
21  // EXEMPLE D'APPEL (avec curl ou Postman) :
22  // POST http://localhost:3000/etudiants
23  // Body: { "nom": "Alice", "email": "alice@test.com", "age": 22,
        "filiere": "Info" }
```

READ - GET /etudiants

```
// Récupérer tous les étudiants (avec filtres optionnels)
app.get('/etudiants', async (req, res) => {
    try {
        // req.query = paramètres URL (?filiere=Info&age=22)
        const filtre = {};

        if (req.query.filiere) {
            filtre.filiere = req.query.filiere;  // Filtre par filière
        }
        if (req.query.age) {
            filtre.age = parseInt(req.query.age);  // Filtre par âge
        }

        // find() avec filtre dynamique
        const etudiants = await Etudiant.find(filtre)
            .select('-__v')           // Exclure champ version mongoose
            .sort({ nom: 1 })         // Tri alphabétique
            .limit(50);               // Limiter à 50 résultats

```

```
            .limit(50);               // Limiter à 50 résultats

        res.json({
            count: etudiants.length,
            data: etudiants
        });
    } catch (erreur) {
        res.status(500).json({ erreur: erreur.message });
    }
});

// Récupérer un étudiant par ID
app.get('/etudiants/:id', async (req, res) => {
    try {
        // req.params.id = valeur dans l'URL
        const etudiant = await Etudiant.findById(req.params.id);

        if (!etudiant) {
            return res.status(404).json({ message: 'Étudiant non trouvé' });
        }
```

```
        }

        res.json(etudiant);
    } catch (erreur) {
        // 400 si ID mal formaté (pas un ObjectId valide)
        res.status(400).json({ erreur: 'ID invalide' });
    }
});
```

UPDATE - PUT /etudiants/:id

```
// Modifier complètement un étudiant (PUT)
app.put('/etudiants/:id', async (req, res) => {
    try {
        // findByIdAndUpdate(id, nouvellesValeurs, options)
        const etudiant = await Etudiant.findByIdAndUpdate(
            req.params.id,
            req.body,           // Nouvelles valeurs
            {
                new: true,      // Retourner le document MODIFIÉ
                runValidators: true  // Valider selon le schéma
            }
        );

        if (!etudiant) {
            return res.status(404).json({ message: 'Étudiant non
                trouvé' });
        }

        res.json({
            message: 'Étudiant mis à jour',
            data: etudiant
```

```
            data: etudiant
        });
    } catch (erreur) {
        res.status(400).json({ erreur: erreur.message });
    }
});

// Modification partielle (PATCH) - plus courant
app.patch('/etudiants/:id', async (req, res) => {
    try {
        const etudiant = await Etudiant.findByIdAndUpdate(
            req.params.id,
            { $set: req.body },  // $set pour ne modifier que les champs fournis
            { new: true }
        );
        res.json(etudiant);
    } catch (erreur) {
        res.status(400).json({ erreur: erreur.message });
    }
});
```

DELETE - DELETE /etudiants/:id

```
// Supprimer un étudiant
app.delete('/etudiants/:id', async (req, res) => {
    try {
        const etudiant = await Etudiant.findByIdAndDelete(req.params.id);

        if (!etudiant) {
            return res.status(404).json({ message: 'Étudiant non trouvé' });
        }

        // 204 No Content ou 200 avec message
        res.json({ message: 'Étudiant supprimé', data: etudiant });
    } catch (erreur) {
        res.status(400).json({ erreur: erreur.message });
    }
});
```

4.4 Route avec Aggregation

```
// GET /stats/filieres - Stats agrégées
app.get('/stats/filieres', async (req, res) => {
    try {
        const stats = await Etudiant.aggregate([
            {
                $group: {
                    _id: "$filiere",
                    total: { $sum: 1 },
                    ageMoyen: { $avg: "$age" },
                    moyenneGenerale: { $avg: { $avg: "$notes" } }
                }
            },
            { $sort: { total: -1 } }
        ]);

        res.json(stats);
    } catch (erreur) {
        res.status(500).json({ erreur: erreur.message });
    }
});
```

4.5 Codes HTTP & Bonnes Pratiques

Code	Usage	Exemple
200	OK	GET réussi
201	Created	POST réussi
400	Bad Request	Données invalides
404	Not Found	Ressource inexistante
500	Server Error	Erreur serveur

```
// BONNES PRATIQUES :

// 1. Toujours utiliser async/await avec try/catch
// 2. Valider les entrées (mongoose le fait via le schéma)
// 3. Gérer les cas null (404 si non trouvé)
// 4. Utiliser les codes HTTP appropriés
// 5. Ne jamais exposer les détails d'erreur internes en production
```